

OSBAMS

Open-Source Battery Assessment and Management System

Joe Le — Computer Engineering
Fall 2026 (August 24 – December 11, 2026)

School of Engineering, San Francisco State University

Abstract

OSBAMS is an open-source hardware/software platform built to figure out how much life is actually left in a reused lithium-ion battery pack, through controlled discharge testing, synchronized electrical and thermal measurement, safety supervision, and data analysis on the host side. The prototype is built around an STM32L476 Nucleo, an INA228 precision current/voltage monitor with an external high-current shunt, a TC74 digital temperature sensor, a redundant ADC for voltage sensing, a hardwired E-stop with contactor disconnect, and an external electronic load. The embedded firmware handles acquisition, sensor validation, state-machine control, watchdog supervision, fault latching, and UART telemetry, while a Python/SQLite desktop application (still in progress) handles live visualization, test storage, battery scoring, and report generation. This isn't meant to replace a calibrated industrial cycler — the priorities are low cost, transparency, modularity, and testing in stages rather than raw precision. At this point in ENGR 696, the custom PCB and most of the hardware are assembled, the firmware builds and flashes without issue, and INA228 telemetry is online. What's left is temperature-sensor bring-up, calibration, safety-chain validation, desktop integration, and eventually controlled battery testing.

1. Introduction

Reused lithium-ion batteries are showing up more and more in mobility, backup power, and energy storage. A pack that's been pulled out of one application can still hold plenty of usable energy, but there's usually no good record of how it was treated — capacity loss, internal resistance, temperature behavior, and general safety condition are all basically unknown. Commercial cyclers can answer these questions reliably, but they're expensive and mostly closed systems, which makes them a poor fit for student use. OSBAMS tries to close that gap with a student-built measurement and safety interface paired with an external electronic load and an open software stack.

1.1 Project Goal

The goal is a portable platform that can safely run a controlled discharge test and measure pack voltage, current, power, temperature, elapsed time, and the ampere-hours and watt-hours delivered along the way. If the E-stop is pressed, a critical sensor fails, a safety limit is exceeded, or the firmware ends up in a state it shouldn't be in, the load path needs to shut off automatically — no exceptions. Test data streams to a host computer and lands in a database, so it can be visualized live and compared against other tests later.

1.2 Significance

There's a lot packed into this project educationally — embedded systems, instrumentation, PCB design, power electronics, software architecture, databases, and safety engineering all show up at some point. Other researchers have already shown that low-cost, open-source battery testers can open up battery research to people who don't have access to lab-grade equipment. OSBAMS takes that same idea and points it at reused-pack assessment specifically, with real attention paid to independent safety controls and redundant measurement rather than treating those as an afterthought. Down the line, once there's a reliable labeled dataset, this could also be a starting point for state-of-health estimation and some machine-learning experiments.

2. Background

2.1 Prior Art in Research and Commercial Development

[1] Su, Xu, and Dong survey state-of-health methods from the cell level up to the pack level, sorting approaches into direct measurement, model-based, data-driven, and hybrid categories. It's part of why OSBAMS starts with trustworthy direct measurement rather than jumping straight to a fancier health-estimation model.

[2] Propp et al. show that a programmable battery cycler can be built for a lot less money once the goal shifts from replacing top-tier lab equipment to just supporting research and education. Their focus on validation is part of the reasoning behind OSBAMS's own emphasis on calibration and staged testing.

[3] Mestre et al.'s open Python/Raspberry Pi tester makes the case for keeping instrument control and measurement separate from an open host-software layer. They also successfully characterized the state of health of LiFePO₄ cells pulled from an e-scooter pack — which is essentially the same motivation behind OSBAMS.

[4] Saletti et al. describe an open-hardware battery/BMS test platform built around safe functional testing, without exposing development to real battery hazards along the way. That's basically the same philosophy behind OSBAMS's staged, low-voltage-first, hardware-in-the-loop approach.

[5] Ng et al. use coulomb counting on measured current and voltage, through controlled charge/discharge experiments, to estimate SOC and SOH. Their measurement chain is basically the model for how OSBAMS accumulates ampere-hours and watt-hours from calibrated current and voltage samples.

Established commercial cyclers obviously win on precision, multi-channel scalability, automated charge/discharge profiles, and calibration support — no argument there. OSBAMS is deliberately narrower: one student-built channel, focused on transparent design, safety, learning, and diagnosing reused packs. The contribution here is integration and validation, not a new SOH algorithm.

2.2 Prior Experience and Project Feasibility

I'm building this alone, drawing on coursework and hands-on experience with embedded programming, electronics assembly, battery systems, and hardware debugging. The STM32L476 has plenty of ADC, GPIO, timers, I²C, UART, and watchdog resources for what this needs. The external electronic load handles the actual power dissipation, which frees the custom PCB to focus purely on sensing, safety, and control. Because the design leans on commercially available components and a fabricated PCB rather than anything exotic, it's realistic to finish within an ENGR 696/697 sequence.

3. Design Approach and Architecture

3.1 Alternative Design Approaches and Trade-Offs

Approach	Advantages	Disadvantages	Decision
Commercial battery cycler only	High accuracy; mature software	High cost; closed architecture; less design learning	Not selected as core platform
PC/Raspberry Pi directly controls hardware	Easy GUI/data tools	Weaker deterministic real-time safety; OS dependence	Host used only for UI/data
FPGA-based controller	Deterministic parallel logic	Higher development complexity for this scope	Not necessary
STM32 hardware-software co-design	Real-time control, ADC/I ² C/UART, watchdog, low cost	Requires firmware and hardware validation	Selected

3.2 Proposed Design

OSBAMS High-Level Architecture

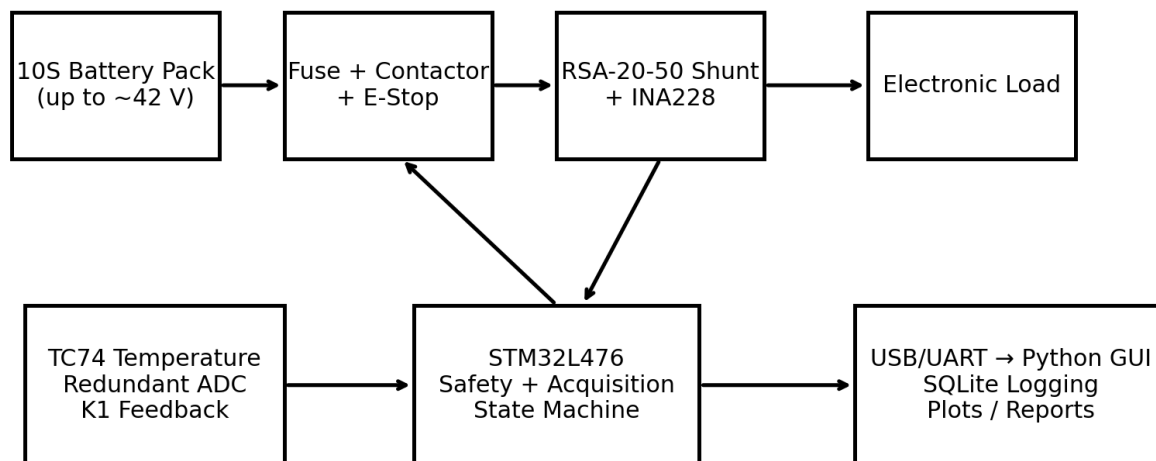


Figure 1. OSBAMS high-level hardware/software architecture.

The high-current path and the logic path are kept physically separate. Battery energy runs through the fuse/disconnect hardware, the contactor, the external shunt, and the electronic load. The STM32 drives the contactor coil through a MOSFET driver, but the E-stop also interrupts that coil path directly in hardware — it doesn't go through the microcontroller at all. INA228 is the primary electrical measurement channel, TC74 covers temperature, and a separate ADC divider serves as an

independent plausibility check on voltage. UART/USB carries structured OSBAMS frames out to the host application.

OSBAMS Firmware / Test Flow

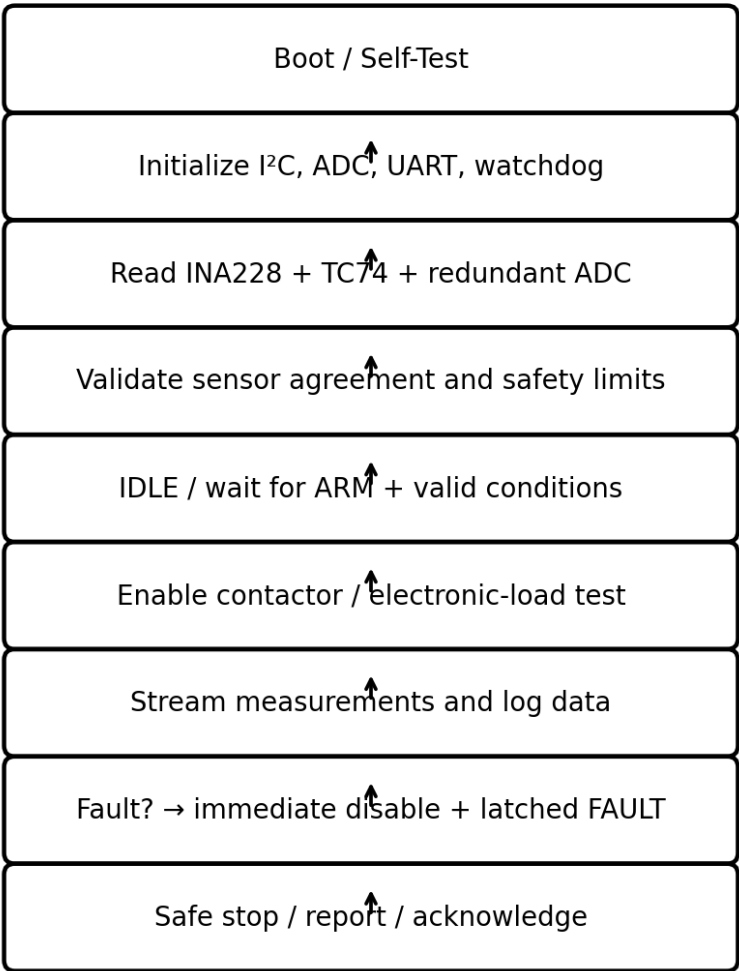


Figure 2. Firmware and test-state flow.

3.3 Design Specifications

Requirement	Target / Design Intent	Verification
Controller	STM32L476 / NUCLEO-L476RG	Build/flash and peripheral tests
Pack voltage	Support approximately 10S Li-ion pack range; independent ADC cross-check	Bench supply + DMM comparison before battery
Current	External shunt + INA228; main current bypasses breakout shunt	Known load current + DMM/load comparison

Temperature	TC74A5-3.3VAT on shared I ² C bus	Address/power/continuity + ambient check
Safety disconnect	Normally-off contactor with hardwired E-stop interruption	Continuity and low-voltage trip test
Firmware safety	Latched FAULT, sensor validation, watchdog, default-off output	Fault injection and reset/recovery tests
Host interface	UART/USB structured frames to Python application	Serial parser and recorded session
Data storage	SQLite test/measurement/fault records	Database query and export test

3.4 Parts and Budget

Part	Qty	Est. Unit	Est. Total	Purpose
NUCLEO-L476RG development board	1	\$25.00	\$25.00	Controller / STM32L476
INA228 current/voltage monitor module	1	\$18.00	\$18.00	Precision bus/current/power sensing
RSA-20-50 external shunt	1	\$15.00	\$15.00	High-current Kelvin shunt
TC74A5-3.3VAT temperature sensor	2	\$1.82	\$3.64	Pack/board temperature
Durakool DG57CM 12 V contactor/relay	1	\$18.00	\$18.00	Hardware disconnect
IRLZ44N MOSFET + gate resistors	1	\$3.00	\$3.00	Low-side coil driver
E-stop switch + ARM switch	1	\$20.00	\$20.00	Human safety / enable controls
Custom OSBAMS PCB fabrication	5	\$25.00	\$125.00	Control/interface PCB batch

Pololu 5 V regulator	1	\$15.00	\$15.00	Logic supply
12 V XDR DIN-rail supply	1	\$25.00	\$25.00	Control power
15 A fuse, holder, XT60, wiring, terminals	1	\$20.00	\$20.00	Power interconnect/protection
Electronic load (existing/owned)	1	\$0.00	\$0.00	Controlled discharge load
Enclosure, DIN rail, mounting hardware	1	\$25.00	\$25.00	Mechanical integration
Misc. diodes, TVS, capacitors, headers	1	\$20.00	\$20.00	Protection/decoupling/interconnect
Estimated prototype total			\$332.64	

These budget numbers are planning estimates — several of the components were already owned or purchased before the semester started, so the actual incremental cost is lower than the total shown.

3.5 Project Tasks and Timeline

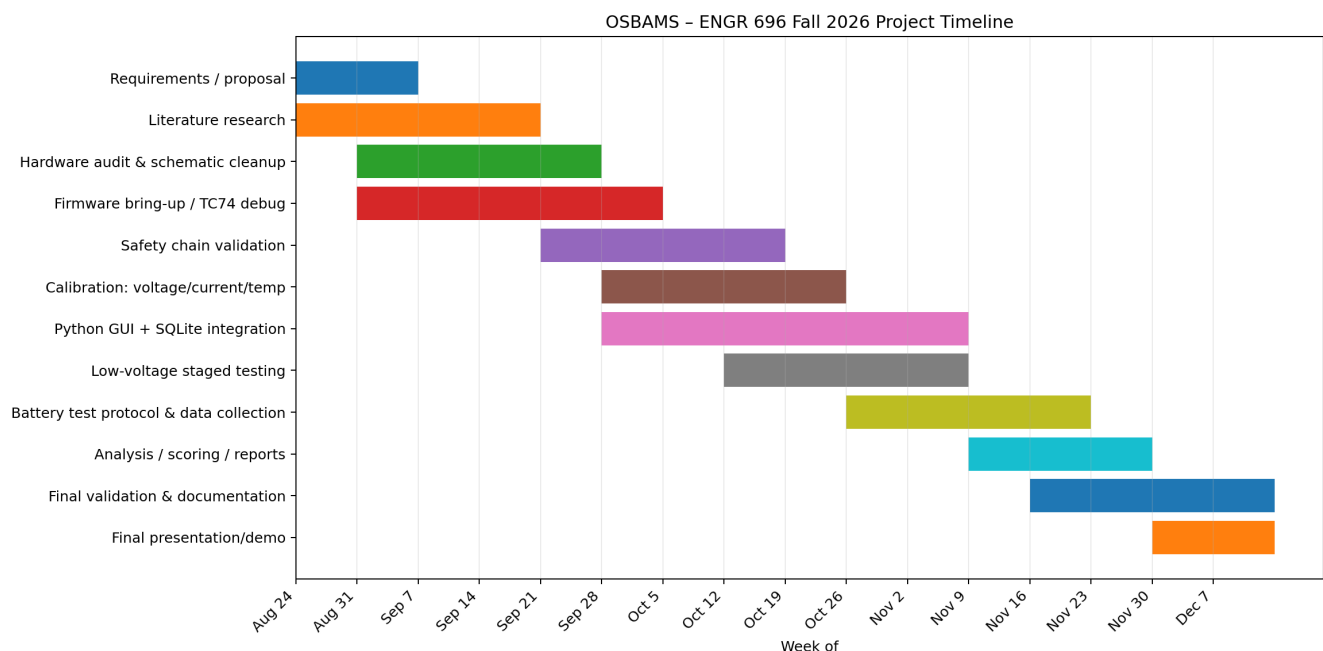


Figure 3. Fall 2026 ENGR 696 timeline.

Since most of the physical prototype already exists, the semester starts with documentation and bring-up rather than building from zero. The critical path is: get sensor communication and power rails solid,

validate the safety chain at low energy, calibrate the measurement channels, integrate the desktop software, and only then move on to actual battery tests. The last few weeks are set aside for repeatability testing, analysis, documentation, and getting the presentation ready.

Current ENGR 696 Status

Done or nearly done: the custom PCB is fabricated and assembled, the Nucleo firmware project is integrated, INA228 communication works, UART boot/status output works, and most of the power, contactor, shunt, E-stop, and electronic-load wiring is in place, along with an initial structure for the Python desktop code. Still in progress: TC74 bring-up, final rail verification, the redundant ADC channel, contactor feedback, calibration, desktop serial/database integration, and staged validation. Full battery testing is being held off on purpose until the system reaches a stable IDLE state with every required sensor and safety function verified.

Sections 4–6 (ENGR 697)

Results and Discussion, Self Assessment, and Final Conclusions will get written once there's actually calibrated battery-test data and repeatability results to report on. They're deliberately left out of ENGR 696 rather than padded out with results that don't exist yet.

References

- [1] Su, L., Xu, Y., & Dong, Z. (2024). State-of-health estimation of lithium-ion batteries: A comprehensive literature review from cell to pack levels. *Energy Conversion and Economics*. <https://doi.org/10.1049/enc2.12125>
- [2] Propp, K., Fotouhi, A., Knap, V., & Auger, D. J. (2016). Design, Build and Validation of a Low-Cost Programmable Battery Cycler. *ECS Transactions*, 74(1), 101–111. <https://doi.org/10.1149/07401.0101ecst>
- [3] Mestre, A. et al. (2018). Open and Flexible Li-ion Battery Tester Based on Python Language and Raspberry Pi. *Electronics*, 7(12), 454. <https://doi.org/10.3390/electronics7120454>
- [4] Saletti, R. et al. (2022). Modular Battery Emulator for Development and Functional Testing of Battery Management Systems: The Cell Emulator. *Electronics*, 11(8), 1215. <https://doi.org/10.3390/electronics11081215>
- [5] Ng, K. S. et al. (2009). Enhanced coulomb counting method for estimating state-of-charge and state-of-health of lithium-ion batteries. *Applied Energy*, 86(9), 1506–1511. <https://doi.org/10.1016/j.apenergy.2008.11.021>